HealthML QA Report

Quality Assurance Report

Project: HealthML
QA Cycle Date: 2026-03-31
Environment: Local development environment
Frontend URL: http://127.0.0.1:5173/
Backend URL: http://127.0.0.1:3001/health

1. Objective

This QA cycle was executed to validate the main HealthML workflow from specialty selection to model results, plus key negative cases such as invalid file upload, schema validation failures, and access control between steps.

2. Scope

Included:

- Step 1: Clinical Context
- Step 2: Data Exploration and Column Mapper
- Step 3: Data Preparation
- Step 4: Model & Parameters
- Step 5: Results
- Step 6: Explainability access behavior
- Step 7: Ethics & Bias availability after training
- Cross-step gating and reset behavior

Excluded:

- Cross-browser testing
- Mobile-specific layout validation
- Performance/load testing
- Real external deployment validation

3. Test Environment

Item	Value
OS	Linux
Browser Runner	Playwright Chromium
Frontend Runtime	Vite
Backend Runtime	Node.js + ts-node

Test Type	Automated E2E smoke and functional QA

Environment Note:

This report reflects the QA execution environment available during this run: local Linux workspace with Playwright Chromium. If the course team requires Windows-only evidence, the same suite should be re-run on the final Windows demo machine and the artifacts attached as an addendum.

4. Execution Summary

Metric	Count
Total Test Cases	10
Passed	10
Failed	0
Blocked	0
Not Run	0

5. Executed Test Cases

Test ID	Module	Scenario	Status
QA-01	Navigation	Block Step 3 before schema validation	Pass
QA-02	End-to-End	Example dataset happy path through training	Pass
QA-03	Step 1	Reset confirmation appears after progress and cancel preserves state	Pass
QA-04	Step 2	Non-CSV upload shows friendly validation error	Pass
QA-05	Step 2	Save Mapping stays blocked until validation and multiple targets are rejected	Pass
QA-06	Navigation	Step 4 blocked until preparation is applied	Pass

QA-07	Step 4	Model visualisation modal opens and closes	Pass
QA-08	Step 5	Compare flow works and does not create duplicate rows	Pass
QA-09	Step 6	Explainability guard shown before training	Pass
QA-10	Step 7	Ethics screen becomes available after training	Pass

6. Requirement Traceability Matrix

Test ID	Sprint 3 Area / Requirement	Acceptance Intent	Result
QA-01	Pipeline gating	Users must not bypass required earlier steps	Pass
QA-02	End-to-end demo workflow	Example-data path must complete from setup to results	Pass
QA-03	Stateful UX safety	Specialty change after progress must trigger reset confirmation	Pass
QA-04	Input validation	Invalid non-CSV upload must be rejected with friendly feedback	Pass
QA-05	Schema validation	Mapping save must require validation and exactly one target column	Pass
QA-06	Preparation gate	Model step must remain blocked until preparation is applied	Pass
QA-07	Step 4 model interaction	Model visualization modal must open and close correctly	Pass
QA-08	Step 5 comparison	Compare flow must	Pass

		add models without duplicate rows	
QA-09	Step 6 gating	Explainability must be blocked until a model has been trained	Pass
QA-10	Step 7 availability	Ethics & Bias screen must be reachable after training	Pass

7. Evidence Register

Evidence ID	Related Tests	Evidence Type	Reference
E-01	QA-01, QA-02, QA-06	Automated Playwright run	npx playwright test executed on 2026-04-14 in local QA environment
E-02	QA-02, QA-08, QA-10	Test implementation	tests/e2e/smoke.spec.ts, tests/e2e/extended.spec.ts
E-03	QA-04	Negative input fixture	tests/fixtures/invalid.txt
E-04	QA-02, QA-07, QA-08, QA-09, QA-10	Shared setup helper	tests/e2e/helpers.ts
E-05	QA execution summary	Console result	Latest Playwright run completed with passing core suite in local environment

Evidence Limitation:

This report currently includes command-based and test-file-based evidence but not screenshot attachments or archived HTML Playwright reports. For a stronger course submission, attach one screenshot or one exported Playwright artifact per major flow.

8. Test Data Register

Data ID	Description	Source	Intended Use
TD-01	Built-in example dataset (~120 synthetic patient rows)	Application default state	Main positive-path validation
TD-02	Invalid text file masquerading as	tests/fixtures/invalid.txt	Non-CSV negative validation

	upload input		
TD-03	In-app mapped schema with outcome as target	Runtime user action in Step 2	Mapping validation and save gating

9. Negative and Resilience Coverage

Covered in this QA cycle:

- Non-CSV file rejection
- Invalid mapper state with multiple target columns
- Blocked navigation before schema validation
- Blocked navigation before preparation
- Explainability blocked before model training

Not yet covered:

- Backend unavailable / server down
- API timeout / slow response behavior
- Malformed CSV with empty headers
- Large but valid CSV near size limit
- Malformed backend JSON or 5xx error rendering
- Numeric parsing edge cases such as mixed string-number columns

10. Performance and Responsiveness Notes

Observed functional indicators from the product and automation flow:

- Training flow completed successfully within normal local demo timing during automated execution
- Results screen rendered after training with metrics and charts visible
- Compare flow updated without duplicate rows

Performance evidence is still missing for a stronger Sprint 3 submission:

- Measured train latency against target
- Auto-retrain debounce timing evidence
- Visualization redraws timing evidence
- Browser-level performance snapshots

Recommendation:

- Add one dedicated performance appendix with measured latency, debounce delay, and any UI timing target already shown inside the product.

11. Key Findings

- The core user journey from Step 1 through Step 5 is functioning correctly in the tested local environment.
- Cross-step gating is enforced as expected for schema validation and preparation requirements.
- Negative validation for invalid upload format and invalid mapper configuration is working.
- Results and downstream educational screens remain reachable after model training.

12. Defect Log

No blocking product defects were confirmed in the executed scope during this QA cycle.

Observations / Known QA Gaps:

Observation ID	Description	Severity	Recommended Action
OBS-01	No screenshot-based evidence bundle is attached yet	Medium	Export screenshots or Playwright HTML report for submission
OBS-02	Environment in this report is Linux; if the final presentation machine is Windows, cross-environment confirmation is still needed	Medium	Re-run suite on final demo machine and append result
OBS-03	Backend failure and malformed CSV resilience paths are not yet automated	Medium	Add negative API and malformed-data tests before final submission

13. Risks and Observations

ID	Type	Description	Severity	Status
R-01	Configuration Risk	Frontend API calls use hard-coded http://localhost:3001 endpoints, which may break in deployed environments without environment-based configuration.	Medium	Open
R-02	Coverage Risk	Current automated coverage validates the main path and selected negative cases but does not yet cover valid CSV upload content variations, backend failure handling, or	Medium	Open

		responsive/mobile behavior.		
R-03	Scope Risk	Sprint evidence in the repository is strongest for Steps 1-5, while Steps 6-7 exist in code but are not equally documented in the visible sprint reports. This may create acceptance ambiguity.	Low	Open

14. Risk Closure Plan

Risk ID	Owner	Action	Target
R-01	Dev Team	Replace hard-coded API base URL with environment configuration	Before final packaging
R-02	QA Team	Add malformed CSV, timeout, backend-down, and larger-file scenarios	Before final grading submission
R-03	Project Team	Add short evidence note clarifying whether Steps 6-7 are Sprint 3 stretch scope or post-sprint additions	Before review/demo

15. Conclusion

Based on the executed QA scope, the HealthML application is stable for its primary demo workflow. The tested user journey is ready for demonstration in a local environment.

Overall Status: Pass

Recommendation: Ready for demo, but the report should be submitted as QA v1.1 with traceability, evidence, and risk notes included. Follow-up QA is still recommended for broader CSV variants, deployment configuration, backend-failure behavior, and performance evidence.